

Soft Actor-Critic With Integer Actions

Ting-Han Fan^{1,2} and Yubo Wang²

Abstract—Reinforcement learning is well-studied under discrete actions. Integer actions setting is popular in the industry yet still challenging due to its high dimensionality. To this end, we study reinforcement learning under integer actions by incorporating the Soft Actor-Critic (SAC) algorithm with an integer reparameterization. Our key observation for integer actions is that their discrete structure can be simplified using their comparability property. Hence, the proposed integer reparameterization does not need one-hot encoding and is of low dimensionality. Experiments show that the proposed SAC under integer actions is as good as the continuous action version on robot control tasks and outperforms Proximal Policy Optimization on power distribution systems control tasks.

I. INTRODUCTION

Recent breakthroughs of Reinforcement learning (RL) have shown promising results from Atari games [1], MuJoCo continuous control [2], robotics [3], to autonomous driving [4]. However, in industrial scenarios such as power systems [5], resource allocation [6], and scheduling [7], the decision variables are often represented by integer quantities. Because the integer variables are not easily differentiable, they raise an issue to the DDPG-style policy gradient [8], [9], [10], a state-of-the-art RL method. While the REINFORCE-based methods [11], [12], [13] are still alternatives to the integer problem, they tend to have high variance [14], [15] and have been shown to be outperformed by the DDPG-style methods [9], [10]. Therefore, it is of practical interest to study methods to parameterize integer random variables and apply them to DDPG-style RL methods.

Gumbel-Softmax is a state-of-the-art reparameterization method for the discrete random variables [16], [17] based on the Gumbel-Max trick [18]. Combining with the Straight-Through estimators [19], [20], the Straight-Through Gumbel-Softmax parameterization can represent a discrete random variable in the forward pass (of a neural network) and maintain differentiability in the backward pass. Since integer variables admit a discrete structure, it is appealing to apply the Gumbel trick for our integer problems.

Our key insight is that integer and discrete variables are fundamentally different: integer variables are comparable while the discrete variables aren't in general. For example, the elements in the integer set $\{1, 2, 3\}$ are comparable because $1 < 2 < 3$, while the discrete set $\{\text{dog}, \text{cat}, \text{frog}\}$ can't assign such a relation. Recall that the one-hot encoding is required for representing a discrete sample due to its incomparability. An integer set has comparability. Ideally, we only need an

integer, not a one-hot vector, to represent an integer sample. In light of this fact, we propose a reparameterization for an integer random variable: use a linear mapping from a one-hot vector of Straight-Through Gumbel-Softmax to an integer. The mapped integer is random and differentiable (w.r.t. parameters of a neural network) because the one-hot vector is. Therefore, this is a valid reparameterization for integer random variables.

Another advantage of the integer-mapping trick is that the integer space can be mapped back to the ambient space, making the effective dimension irrelevant to the fineness of discretization. For example, suppose the ambient action space is an interval $\mathcal{A} = [-1, 1]$, and the interval is discretized into $N \geq 2$ points. Then the discretized action space $\mathcal{A}_d = \{0, \dots, N-1\}$ is mapped back to the ambient action space using $\mathcal{T}(\mathcal{A}_d) = \mathcal{A}_d \frac{2}{N-1} - 1$. Since $\mathcal{T}(\mathcal{A}_d) \subset [-1, 1]$, the effective dimension of $\mathcal{T}(\mathcal{A}_d)$ remains to be one and irrelevant to the fineness of discretization N .

With (1) the differentiability and (2) the advantage on the effective dimension of action space, the proposed integer reparameterization is particularly useful for the DDPG-style method such as the Soft-Actor Critic (SAC) [9]. Thereby, we propose a variant of SAC under integer actions by incorporating our integer reparameterization into the actor network (policy) with the critic network (Q function) staying the same. This is possible because (1) allows us to differentiate through actions and (2) makes the input/output dimension of the actor and critic networks the same as that of the continuous action version.

We validate the proposed SAC with integer actions on PyBullet robot control [21] and PowerGym power systems control [22]. The results show the proposed SAC under integer actions achieve similar performance as the continuous action counterpart in PyBullet and can outperform the PPO [13] algorithm in PowerGym. Hence, this is a certificate of the effectiveness of our SAC with integer actions.

II. REINFORCEMENT LEARNING

We consider an infinite-horizon Markov Decision Process (MDP) $\langle \mathcal{S}, \mathcal{A}, T, r, \gamma \rangle$. $\mathcal{S}, \mathcal{A}$ are state and action spaces, $r(s, a, s')$ is the reward function, $\gamma \in (0, 1)$ is the discount factor, and $T(s'|s, a)$ is the state transition density. Let the initial state distribution be ρ_0 . The objective of RL is to find a policy π that maximizes the γ -discounted cumulative reward:

$$\max_{\pi} J(\pi) = \max_{\pi} \mathbb{E} \left[\sum_{i=0}^{\infty} \gamma^i r(s_i, a_i) \middle| s_0 \sim \rho_0, \right. \\ \left. a_i \sim \pi(\cdot | s_i), s_{i+1} \sim T(\cdot | s_i, a_i) \right]. \quad (1)$$

¹Department of Electrical and Computer Engineering, Princeton University, Princeton, NJ 08544 tinghanf@princeton.edu

²Siemens Technology, 755 College Road East, Princeton, NJ 08540 yubo.wang@siemens.com

Suppose the policy is parameterized by θ as π_θ . A typical way to optimize Eq. (1) by tuning over θ is to use the policy gradient, which depends on the Q function under π_θ :

$$Q^{\pi_\theta}(s, a) = \mathbb{E} \left[\sum_{i=0}^{\infty} \gamma^i r(s_i, a_i) \middle| s_0 = s, a_0 = a, \right. \\ \left. a_i \sim \pi_\theta(\cdot | s_i), s_{i+1} \sim T(\cdot | s_i, a_i) \right]. \quad (2)$$

With Q^{π_θ} defined above, the policy gradient theorem [23] states that

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{s \sim \rho^\pi, a \sim \pi_\theta} [(\nabla_\theta \ln \pi_\theta(a|s)) Q^{\pi_\theta}(s, a)], \quad (3)$$

where ρ^π is the unnormalized discounted state distribution. Since the expectation of Eq. (3) cannot be exactly computed, the gradient estimation becomes an important problem. We will review this topic specifically on discrete random variables in the next section.

III. GRADIENT ESTIMATION FOR DISCRETE RANDOM VARIABLES

Let D be a random variable from a discrete distribution with $N \geq 2$ elements. Let $S = \{s_1, \dots, s_N\}$ be the support of this discrete distribution. Without loss of generality, suppose the discrete distribution p_θ is parameterized by a neural network with parameter θ . We can write

$$D \sim p_\theta, \quad \sum_{d \in S} p_\theta(d) = 1.$$

Given an objective function $f : S \rightarrow \mathbb{R}$, we minimize

$$\min_\theta \mathbb{E}[f(D)],$$

where the expectation is taken over the random sample D . This can be done by computing some estimates of the gradient $\nabla_\theta \mathbb{E}[f]$, which we will review in this section.

A. REINFORCE Estimator

The REINFORCE estimator [11], [24] utilizes a fact from the chain rule:

$$\nabla_\theta p_\theta(d) = p_\theta(d) \nabla_\theta \ln p_\theta(d)$$

Therefore, $\nabla_\theta \mathbb{E}[f]$ is reformulated as

$$\begin{aligned} \nabla_\theta \mathbb{E}[f] &= \sum_d \nabla_\theta p_\theta(d) f(d) = \sum_d p_\theta(d) \nabla_\theta \ln p_\theta(d) f(d) \\ &= \mathbb{E}[\nabla_\theta \ln p_\theta(D) f(D)], \end{aligned}$$

and the REINFORCE estimator simply takes Eq. (4) as an unbiased estimate of $\nabla_\theta \mathbb{E}[f]$.

$$\nabla_\theta \ln p_\theta(D) f(D) \quad (4)$$

Since the policy gradient Eq. (3) and the REINFORCE estimator Eq. (4) share a similar form, it is tempting to develop a policy gradient algorithm upon the REINFORCE estimator. Examples of such algorithms include TRPO [12] and PPO [13]. With the REINFORCE estimator, TRPO and PPO have high variances and however do not require differentiable random actions. On the contrary, without the REINFORCE

estimator, the DDPG-style methods have lower variances but require differentiable random actions. Hence, to reduce the variance, we want to build models for differentiable random actions and use DDPG-style methods such as the SAC to tackle our integer-constrained problem.

B. Gumbel-Softmax

Gumbel-Softmax [16], [17], [25] provides an alternative solution of discrete random variable modeling. By introducing the softmax relaxation, it is considered to be of lower variances during the gradient computation.

Concretely, define the softmax function $\text{Softmax} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ as $[\text{Softmax}(x)]_i = e^{x_i} / \sum_{j=1}^n e^{x_j}$ and suppose $\text{logit}_\theta \in \mathbb{R}^n$, the unnormalized log probability vector, is parameterized by θ . The Gumbel-Max trick [18] states that

$$D \sim \text{Softmax}(\text{logit}_\theta) \stackrel{d}{=} \arg \max(\text{logit}_\theta + G), \quad (5)$$

where G is an i.i.d. n -dimensional $\text{Gumbel}(0,1)$ vector. $D \sim \text{Softmax}(\text{logit}_\theta)$ means that D is sampled from the discrete probability distribution formed by $\text{Softmax}(\text{logit}_\theta)$. The notation $\stackrel{d}{=}$ means the left and the right sides follow the same distribution.

Since Eq. (5) is not differentiable (both the sampling and the argmax), the Gumbel-Softmax approximates the $\arg \max$ of Eq. (5) using a Softmax function, which is differentiable.

$$D_{\text{GS}} = \text{Softmax}(\text{logit}_\theta + G) \quad (6)$$

Despite bringing back the differentiability, Eq. (6) inevitably introduces a bias since D_{GS} is not an one-hot vector. This can be solved by the Straight-Through estimator [26][lecture 15b]. Let $D_{\text{hot}} = \text{OneHot}(D)$ be the one hot version of D from the $\arg \max$ of Eq. (5). The Straight-Through Gumbel-Softmax [16] introduces a constant shift of D_{GS} :

$$D_{\text{STGS}} = D_{\text{hot}} - [D_{\text{GS}}]_{\text{const}} + D_{\text{GS}}, \quad (7)$$

where $[D_{\text{GS}}]_{\text{const}}$ treats D_{GS} as a constant by detaching the dependency on θ during the backward propagation of a neural network. Both D_{hot} and $[D_{\text{GS}}]_{\text{const}}$ are constants for derivatives w.r.t. θ are zeros. Therefore, D_{STGS} is one-hot in the forward propagation (of a neural network) and differentiable in the backward propagation. Redefine the objective function as a differentiable $f : \mathbb{R}^n \rightarrow \mathbb{R}$. The gradient estimate under D_{STGS} becomes

$$\nabla_\theta f(D_{\text{STGS}}) = \left[\frac{\partial f(D_{\text{STGS}})}{\partial \theta_i} \right]_i, \quad (8)$$

where θ_i is the i th element of the model parameter θ . The Straight-Through Gumbel-Softmax estimator is believed to be of lower variance, prevents error in forward propagation, and maintains sparsity [19], [20], [16]. Hence, Eq. (8) has become a popular alternative of Eq. (4), and we would like to incorporate it into the SAC algorithm under a integer constraint setting.

IV. DIFFERENTIABLE RANDOM INTEGERS USING STGS

Although Straight-Through Gumbel-Softmax (STGS) provides a good reparameterization for discrete random variables, directly using discrete variables from STGS to represent integer variables will result in high dimensionality. Thereby, in this section, we will discuss the structure of integer variables and propose a simple linear map trick to reparameterize an integer variable from STGS.

Integer variables usually appear in the industrial setting for the modeling of integer-only or binary decision variables. Integer-only variables can be the number of products or the number of a device's modes. Binary decision variables can be whether to open/close a connection or occupy/release a resource. Despite the discreteness, integer and continuous variables share one thing in common: comparability, or more mathematically, form a totally ordered set. This is the key that makes integer variables distinct from discrete counterparts and allows us to treat integer variables like continuous instead of discrete/categorical variables.

Precisely, a totally ordered set S is a set that satisfies reflexivity ($a \leq a$, $\forall a \in S$), antisymmetry ($a \leq b$ & $b \leq a \Rightarrow a = b$), transitivity ($a \leq b$ & $b \leq c \Rightarrow a \leq c$), and comparability ($\forall a, b \in S$, either $a \leq b$ or $b \leq a$). It is obvious that a continuous variable (e.g. $x \in [-1, 1]$) and a integer variable (e.g. $x \in \{0, 1, 2\}$) range in totally order sets. On the other hand, a discrete/categorical set does not admit a natural comparison; e.g., for $\{\text{dog}, \text{cat}\}$, cat and dog are incomparable. Thus, the comparability fails, and neither do the other properties of the totally ordered set.

Due to the lack of comparability, we usually use an one-hot encoding to represent a discrete/categorical variable; e.g., for $\{\text{dog}, \text{cat}\}$, we have $\text{dog} \rightarrow [1, 0]$ and $\text{cat} \rightarrow [0, 1]$. The encoding dimension inevitably depends on the size of the range, and this becomes problematic when the range is of large size or when there are multiple discrete variables. On the other hand, with the comparability, an integer variable's effective dimension is only one, regardless of the size of its range; e.g., both variables with range $\{0, 1\}$ or $\{0, \dots, 32\}$ need only one dimension to represent. Since the integer variable does not require a one-hot encoding, once being generated, we can treat it like a continuous variable in the subsequent computation. This greatly reduces the computational overhead and is the key to designing a SAC algorithm under integer actions.

The proposed integer reparameterization is as follows. Recall that the Straight-Through Gumbel Softmax, Eq. (7), generates a differentiable one-hot random vector D_{STGS} . Then, $\arg \max_i D_{\text{STGS}}[i]$, the index/argument of 1 in D_{STGS} , is an integer random variable following the distribution formed by $\text{Softmax}(\text{logit}_\theta)$, as the LHS of Eq. (5). Thereby, once establishing an differentiable mapping from D_{STGS} to $\arg \max_i D_{\text{STGS}}[i]$, we can have an differentiable random integer parameterized by θ . Such a differentiable mapping can be constructed by a simple linear map:

$$\text{Ind}_\theta = \langle [0, 1, \dots, n-1], D_{\text{STGS}} \rangle, \quad (9)$$

where $\langle \cdot, \cdot \rangle$ is an inner product and D_{STGS} is assumed to be n -dimensional.

Eq. (9) gives a parameterized random integer Ind_θ with value equaling to $\arg \max_{i \in [0, \dots, n-1]} D_{\text{STGS}}[i]$. We apply this to the SAC under integer actions as follows. Since each integer action is of effective dimension 1 (in a Euclidean space), the action dimension is precisely the number of integer action variables. Thereby, the input/output dimension of the actor network (policy) and the critic network (Q function) is the same as their continuous action counterparts. The only difference lies in the policy network design, where we restrict the output to have an integer structure using Eq. (9) while the continuous action version doesn't have such a restriction.

V. RELATED WORK

Reinforcement learning (RL) with discrete action space has been studied in different aspects over the literature. Due to the high dimensionality of a discretized action space, it has been of great interest to scale RL to large spaces using a neural network. Examples of this line of research include sequentially predicting an action dimension at a time [27], training a Q function for each action dimension [28], and factorizing the action across dimension using the natural ordering between discrete actions [29] or normalizing flows [30], [31]. Various techniques have been proposed in different settings. When the action space admits a continuity structure, a discrete action can be derived by rounding a continuous one [32]. When the action is represented by a one-hot vector, discrete soft actor-critic [33] and Gumbel-Softmax [34] have been taken as key techniques to tackle RL problems.

Our work and the literature share the same motivation as to apply RL to large action spaces. Although our work includes techniques that have been studied (e.g., SAC and Gumbel-Softmax), we reformulate them using a more efficient representation. For example, the one-hot encoding is required in discrete soft actor-critic [33] and Gumbel-Softmax for image-text matching [34]. However, as proposed in Eq. (9), our integer representation is much simpler than a one-hot encoding. The key reason for having such a reduction is that an integer set admits a continuity structure. Although this continuity observation has been used to justify the rounding technique [32], rounding gives only a non-differentiable integer, rather than a differentiable one as required in this work. Hence, our work is different from the literature in the sense that we re-organize the techniques and give a simpler form of differentiable random integers.

VI. EXPERIMENTS AND DISCUSSIONS

We evaluate our integer reparameterization for SAC on two OpenAI Gym environments [35]: PyBullet robot control [21] and PowerGym power network control [22]. The results show the proposed SAC under integer actions achieve similar performance as the continuous action version in Pybullet and outperform PPO [13] algorithm in PowerGym. Hence this shows the effectiveness of the proposed integer reparameterization for SAC algorithm under integer actions.

A. PyBullet Environment

PyBullet is an open-source version of the MuJoCo environment [2] for robot control tasks. Compared to MuJoCo, PyBullet is license-free and is more difficult to solve [36], [37]. Since the actions in PyBullet are continuous and bounded, we can derive the integer action version of PyBullet by discretizing the actions. Hence, we adopt the PyBullet environment to evaluate the performance discrepancy between integer actions and continuous actions for robot control tasks.

Suppose the dimension of a PyBullet continuous action space is K . Because each dimension of a PyBullet action is bounded in an interval $I = [-1, 1]$, we can discretize each dimension as follows. Suppose the interval I is discretized into $N \geq 2$ integers as $I_d = \{0, 1, \dots, N-1\}$. After having an integer action $a \in I_d$, a is embedded into I by

$$\mathcal{T}(a) = a \frac{2}{N-1} - 1 \in I.$$

Such discretization produces a discrete action space with size N^K where K is the euclidean dimension of a PyBullet action space. The size quickly becomes intractable if using a one-hot encoding for the discrete actions. Even if using a multi-hot encoding, the size of a multi-hot vector, NK , is still undesirable for the implementation of the actor and critic networks. On the other hand, if we apply the proposed integer reparameterization, Eq. (9), for each dimension of a PyBullet action, then we recover a K -dimensional integer vector because each integer action is of effective dimension 1. Hence, the discretized environment is highly nontrivial under one-hot and multi-hot encodings but is feasible under the proposed integer reparameterization.

Environment	Reacher	Hopper	HalfCheetah	Walker2D
Temperature	0.005	0.05	0.02	0.06

TABLE I

TEMPERATURE PARAMETERS FOR PYBULLET ENVIRONMENTS.

We run SAC in continuous or discretized (integer) actions in four PyBullet environments: Reacher, Hopper, HalfCheetah, Walker2D. Both (cont. or int.) use the same temperature parameters from Table I. Figure 1 shows SAC with continuous or integer actions share very similar performances. Note SAC with continuous actions is known to perform well on PyBullet tasks [38], [37] in the sense that it outperforms PPO. Therefore, this shows our SAC with integer actions is among the best and certifies the usefulness of the proposed integer reparameterization trick.

B. PowerGym Environment

We demonstrate a use case of the proposed integer constraint SAC in an industrial setting. PowerGym [22] is an open-source reinforcement learning environment for Volt-Var control in power distribution systems. By controlling the circuit elements such as capacitors, regulators, and batteries, the Volt-Var control aims to minimize the voltage violations, control loss, and power loss subject to physical networked constraints and device constraints.

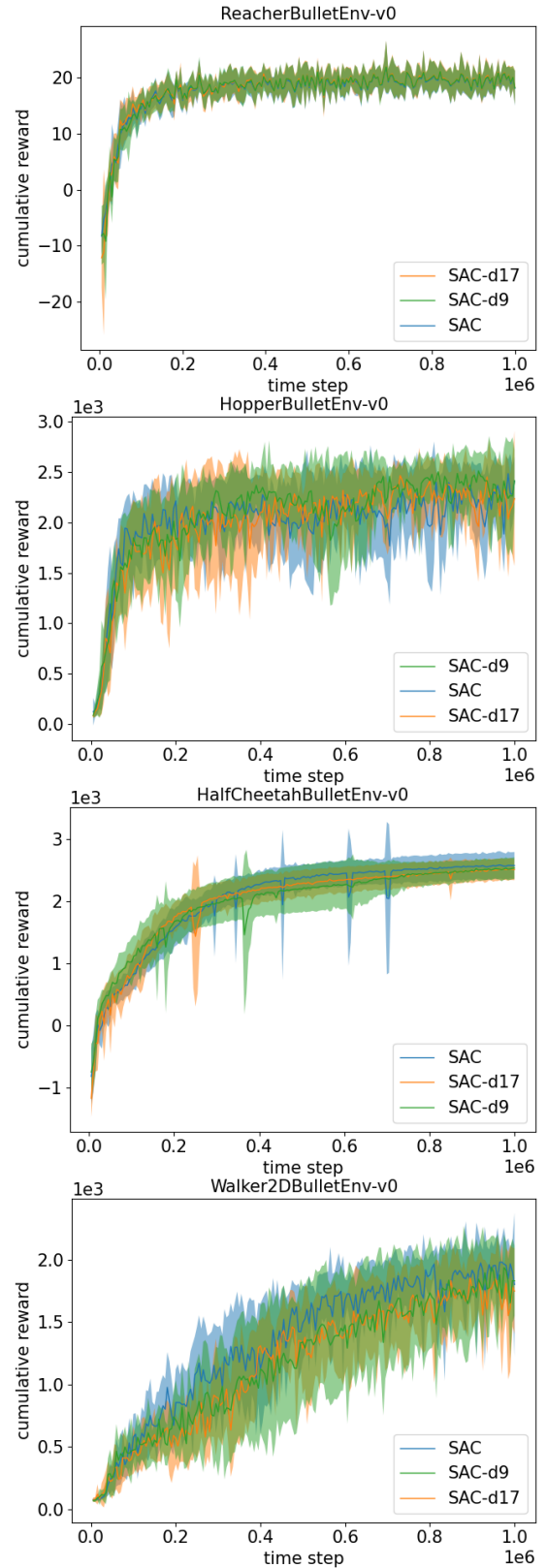


Fig. 1. Experiments on PyBullet. The solid lines are the averages and the shaded regions are the standard deviation. d9 and d17 mean to discretize each dimension of the action space into 9 and 17 integers, respectively.

The device constraints restrict the decisions on the con-

trollers to be integers. As shown in Table II, a capacitor's status is binary (on/off), a regulator's tap number is an integer in finite range, and a battery's discharge power is also represented by an integer. Hence, PowerGym is a good example of a practical environment with integer actions.

Variable	Type	Range
Capacitor status	disc.	$\{0, 1\}$
Regulator tap number	disc.	$\{0, \dots, N_{\text{reg_act}} - 1\}$
Discharge power	disc.	$\{0, \dots, N_{\text{bat_act}} - 1\}$

TABLE II

ACTION SPACE. $N_{\text{REG_ACT}}$ AND $N_{\text{BAT_ACT}}$ ARE 33 BY DEFAULT.

System	# Caps	# Regs	# Bats
13Bus	2	3	1
34Bus	2	6	2
123Bus	4	7	4
8500Node	10	12	10

TABLE III

SPECIFICATIONS OF POWERGYM ENVIRONMENT.

We evaluate SAC with integer reparameterization on four PowerGym environments, as shown in Table III. Since the actions are pure integers, we cannot have a comparison between continuous and discretized actions as Figure 1. Instead, take PPO [13] as a benchmark against our SAC under integer actions. Note that the literature always implements SAC with reparameterization and PPO without reparameterization [9], [38], [37]. This is because PPO is based on the REINFORCE estimator [11] and does not need differentiable random actions. Following this convention, in this experiment, SAC is equipped with integer reparameterization while PPO is using REINFORCE.

Figure 2 shows that SAC outperforms PPO in all four environments. This demonstrates the usefulness of the proposed integer reparameterization for SAC. In addition, because PPO does a clipping at its policy gradient while SAC doesn't, PPO starts near the random policy and improves steadily but slowly. On the other hand, SAC is susceptible to the accuracy of the Q function, so it deviates from the random policy in the early steps but improves quickly as the Q function becomes more accurate, which explains the swing of cumulative reward in the 8500Node experiment.

VII. CONCLUSION

Motivated by industrial scenarios, we study reinforcement learning under integer actions. We approach the problem by incorporating the Soft Actor-Critic (SAC) algorithm with an integer reparameterization on actions. Although the Gumbel-Softmax is capable of modeling discrete random variables, the discreteness of the integer actions can be simplified using the comparability property. The proposed integer reparameterization is based on Gumbel-Softmax followed by a linear mapping for output dimension reduction. Experiments show that the proposed SAC under integer actions is as

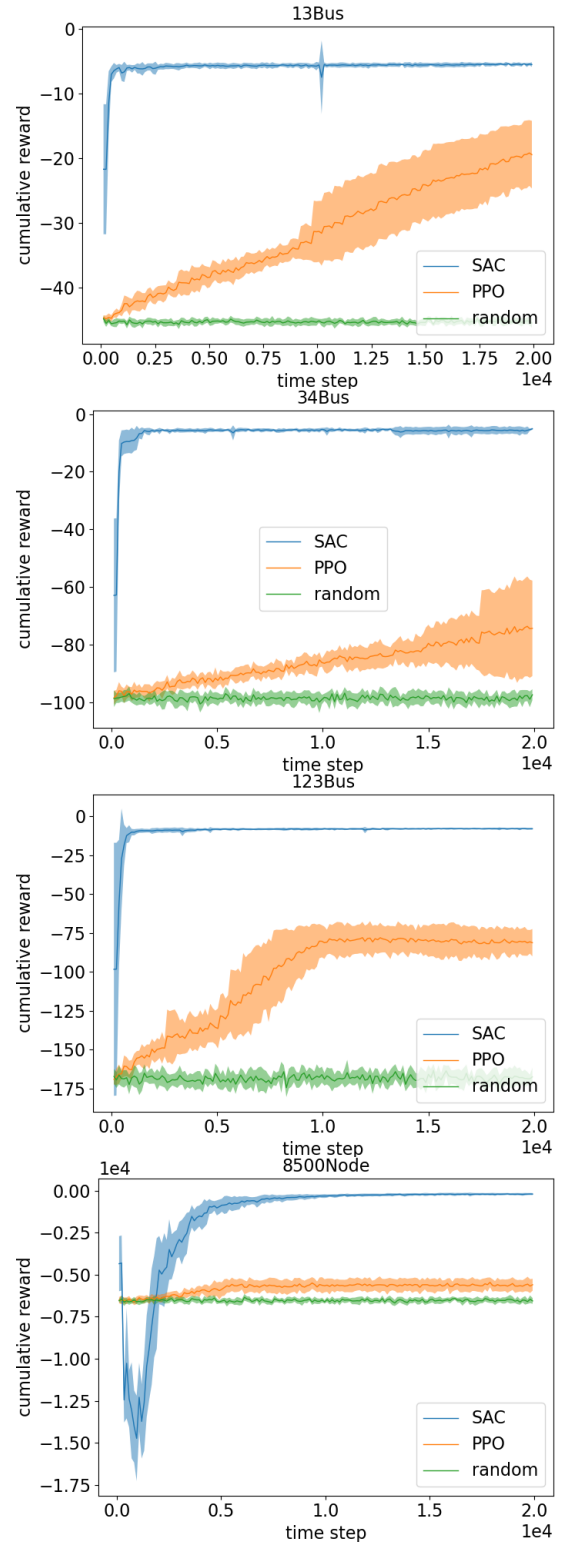


Fig. 2. Experiments on PowerGym. "random" denotes a uniform random policy that takes actions uniformly at random from the action space.

good as the continuous action version on robot control tasks and can outperform Proximal Policy Optimization on power distribution tasks. Hence, it suggests the usefulness of the proposed integer reparameterization on the SAC algorithm.

REFERENCES

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.
- [2] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2012, pp. 5026–5033.
- [3] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *The Journal of Machine Learning Research*, vol. 17, no. 1, pp. 1334–1373, 2016.
- [4] E. Leurent, "An environment for autonomous driving decision-making," <https://github.com/eleurent/highway-env>, 2018.
- [5] A. Marot, B. Donnot, G. Dulac-Arnold, A. Kelly, A. O'Sullivan, J. Viebahn, M. Awad, I. Guyon, P. Panciatichi, and C. Romero, "Learning to run a power network challenge: a retrospective analysis," *arXiv preprint arXiv:2103.03104*, 2021.
- [6] A. A. Zoltners and P. Sinha, "Integer programming models for sales resource allocation," *Management Science*, vol. 26, no. 3, pp. 242–260, 1980.
- [7] Y.-C. Wang and J. M. Usher, "Application of reinforcement learning for agent-based production scheduling," *Engineering Applications of Artificial Intelligence*, vol. 18, no. 1, pp. 73–82, 2005.
- [8] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *International conference on machine learning*. PMLR, 2014, pp. 387–395.
- [9] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning*. PMLR, 2018, pp. 1861–1870.
- [10] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International Conference on Machine Learning*. PMLR, 2018, pp. 1587–1596.
- [11] P. W. Glynn, "Likelihood ratio gradient estimation for stochastic systems," *Communications of the ACM*, vol. 33, no. 10, pp. 75–84, 1990.
- [12] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *International conference on machine learning*. PMLR, 2015, pp. 1889–1897.
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [14] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2016.
- [15] C. Wu, A. Rajeswaran, Y. Duan, V. Kumar, A. M. Bayen, S. Kakade, I. Mordatch, and P. Abbeel, "Variance reduction for policy gradient with action-dependent factorized baselines," in *International Conference on Learning Representations*, 2018.
- [16] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," in *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.
- [17] C. J. Maddison, A. Mnih, and Y. W. Teh, "The concrete distribution: A continuous relaxation of discrete random variables," in *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.
- [18] C. J. Maddison, D. Tarlow, and T. Minka, "A* sampling," in *Advances in Neural Information Processing Systems*, Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Q. Weinberger, Eds., vol. 27. Curran Associates, Inc., 2014.
- [19] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv preprint arXiv:1308.3432*, 2013.
- [20] J. Chung, S. Ahn, and Y. Bengio, "Hierarchical multiscale recurrent neural networks," in *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.
- [21] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning," <http://pybullet.org>, 2016–2021.
- [22] T.-H. Fan, X. Y. Lee, and Y. Wang, "Powergym: A reinforcement learning environment for volt-var control in power distribution systems," *arXiv preprint arXiv:2109.03970*, 2021.
- [23] R. S. Sutton, D. A. McAllester, S. P. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Advances in neural information processing systems*, 2000, pp. 1057–1063.
- [24] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine learning*, vol. 8, no. 3, pp. 229–256, 1992.
- [25] M. B. Paulus, C. J. Maddison, and A. Krause, "Rao-blackwellizing the straight-through gumbel-softmax gradient estimator," in *International Conference on Learning Representations*, 2021.
- [26] G. Hinton, N. Srivastava, and K. Swersky, "Neural networks for machine learning," *Coursera, video lectures*, 2012.
- [27] L. Metz, J. Ibarz, N. Jaitly, and J. Davidson, "Discrete sequential prediction of continuous actions for deep rl," *arXiv preprint arXiv:1705.05035*, 2017.
- [28] A. Tavakoli, F. Pardo, and P. Kormushev, "Action branching architectures for deep reinforcement learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, 2018.
- [29] Y. Tang and S. Agrawal, "Discretizing continuous action space for on-policy optimization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 2020, pp. 5981–5988.
- [30] —, "Boosting trust region policy optimization by normalizing flows policy," *arXiv preprint arXiv:1809.10326*, 2018.
- [31] O. Delalleau, M. Peter, E. Alonso, and A. Logut, "Discrete and continuous action representation for practical rl in video games," *arXiv preprint arXiv:1912.11077*, 2019.
- [32] H. van Hasselt and M. A. Wiering, "Using continuous action spaces to solve discrete problems," in *2009 International Joint Conference on Neural Networks*, 2009, pp. 1149–1156.
- [33] P. Christodoulou, "Soft actor-critic for discrete action settings," *arXiv preprint arXiv:1910.07207*, 2019.
- [34] S. Yan, L. Yu, and Y. Xie, "Discrete-continuous action space policy gradient-based attention for image-text matching," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 8096–8105.
- [35] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," *arXiv preprint arXiv:1606.01540*, 2016.
- [36] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, "Sim-to-real: Learning agile locomotion for quadruped robots," in *Robotics: Science and Systems (RSS)*, 2018.
- [37] A. Fayad and M. Ibrahim, "Behavior-guided actor-critic: Improving exploration via learning policy behavior representation for deep reinforcement learning," *arXiv preprint arXiv:2104.04424*, 2021.
- [38] A. Raffin, J. Kober, and F. Stulp, "Smooth exploration for robotic reinforcement learning," *arXiv preprint arXiv:2005.05719*, 2021.